围棋大作业报告

一、算法设计思路和实现细节

1.Random AI

这一部分的目的是基于随机落子（但需满足规则）的基础围棋 AI 用于验证规则调用和基础设施正常工作，从所有合法棋步中均匀随机选择正常落子、停一手和认输。

本作业的 Random 实现在 random_agent.py 文件中，主体是 RandomAgent 类。其 select_move(game_state) 方法的实现分为两步：

（1）调用 game_state.legal_moves() 获取当前全部合法动作；

（2）调用 random.choice(legal_moves) 在合法动作中等概率随机选择一个并返回。

Random 算法的优点是实现简单、运行速度快、便于验证系统流程；缺点是没有任何策略性，无法主动判断局面优劣，因此棋力较弱、结果波动较大。

2.MCTS AI

这一部分的目的是在 5×5 棋盘上实现基于蒙特卡洛树搜索的围棋 AI，使其在有限时间内完成相对更合理的落子。包含四个标准步骤：选择、扩展、模拟、反向传播，并在模拟阶段加入了两种提速思路（限制深度、启发式走子）来提高效率。

具体实现细节如下：

（1）初始化阶段：self.move = game_state.last_move 获取到达当前节点时走的那一步，用于最终返回落子；self.untried_moves 用于当前局面下尚未扩展的合法落子列表，这里只保留 is_play，不把 pass/resign 加入树扩展，目的是尽量优化性能。

（2）实现价值计算：每个节点保存 visit_count 和 value_sum，节点平均价值等于 value_sum / visit_count，这样就能用历史模拟结果衡量这个节点好不好。

（3）选择阶段：best_child 使用 UCT 思路在已有子节点里选择下一步要深入的节点。对还未访问的节点直接给无穷大分数，保证每个分支至少会被探索一次；对已访问节点则根据 UCT 思路综合计算历史平均价值和探索奖励两个部分，避免只贪心也避免盲目随机。

（4）扩展阶段：expand 从 self.untried_moves 中随机取一个动作并移除，然后对该动作执行 apply_move 得到 next_state，创建新子节点并加入 self.children。这样可以逐步把还没试过的动作加入搜索树，避免一开始就把所有分支全部展开导致开销过大。

（5）模拟阶段：_simulate(game_state) 从当前节点出发做快速模拟。这里用了两点

优化：一是限制最大模拟深度 max_depth，避免单次模拟过慢；二是调用 _pick_rollout_move 在候选落子中做简单启发式选择，考虑吃子收益和中心距离，比完全随机更稳定。

（6）反向传播阶段：backup(value) 将模拟结果从当前节点一路回传到根节点。每经过一个节点就更新 visit_count 和 value_sum；由于双方轮流落子，向父节点回传时使用 1 - value 做视角切换，使统计值始终和当前走子方一致。

（7）主循环阶段：select_move(game_state)先创建根节点 root，然后重复 num_rounds 轮选择→扩展→模拟→回传。循环结束后调用_select_best_move(root)，按子节点访问次数最大原则返回最终落子，目的是让决策更稳健、减少偶然模拟带来的波动。

（8）最终决策阶段：_select_best_move(root) 在 root.children 中选择 visit_count 最大的子节点并返回其 move；如果没有可用子节点则返回 pass。相比直接按一次模拟分数选择，按访问次数选择通常更能反映多轮搜索后的整体统计结果。

3.Minimax AI

（1）初始化阶段：在 MinimaxAgent.__init__中设置 max_depth 作为搜索深度上限，evaluator 作为局面评估函数（未传入时使用默认评估_default_evaluator），并创建 GameResultCache 用于缓存已评估局面，减少重复搜索开销。

（2）入口决策阶段：select_move(game_state)先生成合法落子（仅保留 is_play），若没有可下动作则返回 pass。之后根据当前执子方分别做最大化/最小化搜索：黑方按取最大值挑选最佳动作，白方按取最小值挑选最佳动作。根节点逐个尝试候选落子，并调用 alphabeta 计算每个落子的估值，最终返回最优动作。

（3）基础递归阶段：minimax(game_state, depth, maximizing_player)在终止条件（depth==0 或终局）下直接返回评估值；否则递归遍历下一层动作。若是最大化层就取所有子节点中的最大值，若是最小化层就取最小值。

（4）剪枝优化阶段：alphabeta(game_state, depth, alpha, beta, maximizing_player)在 Minimax 基础上加入剪枝条件。最大化层中若当前值已满足 alpha >= beta 则停止后续分支；最小化层中若 beta <= alpha 也立即剪枝。这样能明显减少不必要搜索，提高同等深度下的运行速度。

（5）缓存阶段：在 alphabeta 中使用 state_key = (zobrist_hash, next_player)查询缓存，若命中且缓存深度不小于当前需求深度，则直接返回缓存值。搜索完成后把结果通过 cache.put(state_key, depth, value, flag)写回，避免重复评估同一局面，提升效率与稳定性。

（6）评估函数阶段_default_evaluator(game_state)在终局时给出大幅胜负分（黑胜正高分、白胜负高分）；非终局时基于“棋子数量差+气数差”做启发式评分。

（7）走子排序阶段：_get_ordered_moves(game_state)对候选动作做简单排序（优先靠近棋盘中心），目的是让更可能较优的动作先被搜索，从而让 Alpha-Beta 更早触发剪枝，提高剪枝效率。

（8）缓存容器实现：GameResultCache 提供 get/put 两个接口；put 中使用深度优先替换策略，保证缓存中尽量保留价值更高的搜索结果。

4.图形化界面

本作业图形界面使用 Python 内置的 Tkinter 实现，文件为 interaction.py。

（1）界面结构：主要分为顶部控制区、中央棋盘区、底部状态区三部分。顶部包含 AI 模式选择（random/mcts/minimax）和功能按钮（停一手、认输、悔棋、新游戏）；中央使用 Canvas 绘制 5×5 棋盘和棋子；底部显示当前回合、AI 状态和终局计分信息。

（2）人机交互流程：用户通过鼠标点击棋盘交叉点落子，程序会先做坐标映射，再调用规则检查合法性。若合法则更新局面并重绘棋盘；若非法则在状态栏提示“非法落子”。当轮到 AI 时，自动调用对应智能体进行选点并执行落子。

（3）AI 模式切换：界面支持 random、mcts、minimax 三种模式下拉切换。切换后会重建对应 AI 并自动开启新局，确保模式切换后的对局状态一致。

（4）对局控制功能：实现了停一手、认输、新游戏、悔棋四个基础功能。

（5）终局与信息展示：对局结束后，界面显示胜负结果和计分信息，并在正常终局时展示贴目后的分数计算；若为认输结束，则明确标注“对手认输”。

二、MCTS 优化方法

1.优化目标

标准 MCTS 在每次落子时需要做很多次模拟。如果模拟完全随机、并且每次都走到很深，虽然逻辑简单，但会带来单步耗时偏长、模拟结果噪声大，决策不够稳定等问题。因此，本作业在保持 MCTS 基本流程不变（选择、扩展、模拟、回传）的前提下，加入了两种轻量优化。

2.本作业采用的两种优化方法

（1）限制模拟深度：在 rollout 阶段设置最大深度（例如 24 步），达到深度上限后停止模拟，并用一个快速评估方式估计当前局面优劣。作用是减少单轮模拟时间，提升

整体响应速度。

（2）启发式模拟走子：在 rollout 阶段不采用完全随机走子，而是对候选落子做简单打分，再从前几名里随机挑选。本实现中主要考虑：是否可能吃子、是否更靠近棋盘中心。作用是减少无意义模拟，提高每轮模拟的有效性。

三、不同算法（MCTS、Minimax）的性能分析

在本项目中，MCTS 属于基于采样的统计搜索，Minimax 属于基于博弈树展开的确定性搜索。具体来说，MCTS 在每一步决策时会先建立根节点，然后重复执行“择—扩展—模拟—反向传播”四个步骤，最终按照子节点访问次数选择落子；在本项目测试脚本中，MCTS 默认使用 num_rounds=1000，并在模拟阶段设置了最大深度 24，同时通过优先考虑吃子和靠近中心的位置来优化 rollout。相比之下，Minimax 则在 max_depth=3 的固定深度下进行递归搜索，并结合 Alpha-Beta 剪枝、局面缓存和候选着法排序来减少无效分支，其默认评估函数主要依据“黑白棋子数差”和“气数差”来估计局面优劣。

从性能特点来看，Minimax 的优点是搜索逻辑清晰、结果可解释性强。在搜索深度较浅、棋盘规模较小的情况下，它能够比较稳定地比较当前局面的直接得失；再加上 Alpha-Beta 剪枝、Zobrist 哈希缓存和中心优先排序，实际运行效率比朴素 minimax 已有明显提升。它的不足也比较明显，由于搜索深度固定为 3，评估函数又相对简单，算法对中长期局面结构的判断能力有限。

MCTS 的优势则在于，它不依赖一个非常精确的静态评估函数，而是通过大量模拟来近似判断哪一步更有前景，因此在围棋这种分支多、局面复杂的游戏中更有适应性。尤其在本项目里，MCTS 还加入了“限制模拟深度”和“启发式选择 rollout 走法”两种优化，这使它相较于完全随机模拟更容易得到较合理的落子结果。与此同时，MCTS 也有自己的局限，当前实现最终只按照访问次数选点，模拟阶段使用的也是较简单的启发式，因此在某些极端局面下仍会出现判断偏差。

四、图形界面设计说明

界面实现了人机对弈的基本功能：（1）棋盘和棋子显示；（2）鼠标点击落子；（3）停一手、认输、悔棋、新游戏；（4）可切换三种 AI 模式（random、mcts、minimax）；（5）状态栏显示当前回合和终局信息。

五、测试结果与截图

测试 1 python play.py --agent1 random --agent2 random --size 5

```
========== 统计 ==========
对局数: 1
黑方 (random) 胜: 1
白方 (random) 胜: 0
平局: 0
平均步数: 10.0
平均用时: 0.18s
```

测试 2 python play.py --agent1 mcts --agent2 random --size 5

```
========== 统计 ==========
对局数: 1
黑方 (mcts) 胜: 1
白方 (random) 胜: 0
平局: 0
平均步数: 48.0
平均用时: 9.18s
```

测试 3 python play.py --agent1 mcts --agent2 mcts --size 5 --games 10

```
========== 统计 ==========
对局数: 10
黑方 (mcts) 胜: 0
白方 (mcts) 胜: 0
平局: 10
平均步数: 51.0
平均用时: 30.74s
```

测试 4 python play.py --agent1 minimax --agent2 random --size 5

```
========== 统计 ==========
对局数: 1
黑方 (minimax) 胜: 1
白方 (random) 胜: 0
平局: 0
平均步数: 12.0
平均用时: 1.09s
```

测试 5 python play.py --agent1 minimax --agent2 mcts --size 5

```
========== 统计 ==========
对局数: 1
黑方 (minimax) 胜: 0
白方 (mcts) 胜: 0
平局: 1
平均步数: 51.0
平均用时: 9.93s
```

六、与 AlphaGo 和 AlphaZero 的对比思考

与 AlphaGo 和 AlphaZero 相比，本项目实现的围棋 AI 仍然属于较为基础的搜索式方法。AlphaGo 的核心是在蒙特卡洛树搜索的基础上，引入策略网络与价值网络来分别辅助落子选择和局面评估，从而显著提高搜索效率与决策质量；而 AlphaZero 则进一步取消了对人类棋谱的依赖，主要通过自我对弈和强化学习不断提升自身水平，实现了更强的泛化能力和更高的竞技表现。相比之下，本项目中的 Random、Minimax 和 MCTS 智能体都没有结合深度神经网络，也不具备自动学习能力，主要依赖规则、搜索和随机模拟完成决策，因此在搜索深度、局面理解能力和整体棋力上都与现代顶尖围棋 AI 存在明显差距。不过，从教学和实践的角度来看，本项目较好地帮助我理解了围棋 AI 的基本实现逻辑，尤其是博弈树搜索、蒙特卡洛模拟以及不同算法之间的性能差异。

七、AI 辅助使用说明（模型：GPT-5.3-Codex）

在具体实现过程中，AI 的辅助主要用在以下几个方面：（1）帮助我阅读和理解助教提供的围棋规则框架代码，例如 GameState、Move、棋盘状态更新、合法落子判断等核心逻辑；（2）是在 MCTS 和 Minimax 两部分实现时，协助我梳理算法流程，明确选择、扩展、模拟、反向传播以及 Alpha-Beta 剪枝等关键步骤，将我已经掌握的人工智能原理知识转化为编程语言，然后我在展开程序编写；（3）在完成全局编写之后对我的部分代码提出优化的意见，我在此基础上进行更改；（4）在图形界面部分，由于自身完全没有可视化的经验和能力，AI 为我提供了 Tkinter 相关的实现思路和代码参考，包括窗口布局、棋盘绘制、鼠标点击落子、按钮事件绑定、状态栏更新等内容，但这些内容并不是直接照搬使用，是在理解的基础上结合本次作业的功能需求进行取舍、调整和整合，最终形成了现在这份较为完整的人机对弈界面。

使用感受：由于我之前缺乏 Python 编程经验，AI 在一定程度上帮助我完成了从理解语法到实际编写代码的过渡，在此过程中也学习到了很多新的编程知识；AI 提高了开发效率，尤其在结构组织、代码调试方面帮助很大；但 AI 有时候会出现过度的设计，与作业的要求有出入，所以在此过程中需要自己的判断和简化。